

Legacy Frontend Structure & Cleanup Assessment

Executive Summary

The current legacy-version/ UI is a Vite + React SPA (React Query + tRPC v11 + superjson) that was originally designed to run inside a Manus-hosted environment, with OAuth sign-in handled via a proprietary “OAuth portal” and a Manus runtime plugin.

In our current Go-based backend world, that Manus/OAuth integration is both unnecessary and actively harmful: the UI crashes at startup because `getLoginUrl()` eagerly constructs a URL from missing Vite env vars (`VITE_OAUTH_PORTAL_URL`, `VITE_APP_ID`). This ticket focuses on: (1) documenting the legacy frontend in a way that makes cleanup safe, and (2) removing the legacy auth/OAuth codepath so local dev can run without any proprietary dependencies.

This document is “textbook style”: it first explains how the UI is structured and built, then assesses each major subsystem for simplification/removal/rewrite.

Problem Statement

We have two conflicting realities:

1. The legacy UI assumes a Manus OAuth environment (portal URLs, callback endpoints, and runtime conventions).
2. The current product direction is a Go backend (markdown-quizz/) that already operates without real auth (synthetic/admin user for local/dev) and serves the SPA as static assets.

The immediate operational problem is a startup crash in the UI:

- `getLoginUrl` → `useAuth` → page render
- `getLoginUrl` depends on `import.meta.env.VITE_OAUTH_PORTAL_URL` and `VITE_APP_ID`
- When those are unset (local dev), `new URL(...)` throws and React crashes before any rendering.

The longer-term problem is maintainability:

- legacy-version/ contains an entire Express+tRPC backend, MySQL/Drizzle migrations, and Manus SDK integration that we do not run anymore.
- The UI still imports legacy server types (`server/routers.ts`) to get `AppRouter` type information.
- Several dependencies and patterns are Manus-specific and should either be removed or isolated.

Proposed Solution

This ticket (MO-001) proposes a staged cleanup of legacy-version/, starting with a targeted removal of auth/OAuth from the UI:

1. Document the current legacy UI structure, build chain, pages, and major components (Part 1).
2. Assess each subsystem for:
 - simplification (make it smaller / more obvious),
 - removal (delete entirely),
 - or rewrite (replace proprietary dependencies).
3. Implement a “no-auth” frontend behavior:

- remove Manus OAuth URL generation + login redirects,
- remove “Sign In Required” gates,
- keep the SPA fully usable against the Go backend without any special env vars.

Non-goals for this ticket:

- Rewriting the entire UI or migrating component libraries.
- Deleting the legacy server folder (that needs a dedicated plan because the UI currently imports type info from it).
- Replacing tRPC in the frontend.

The only execution work explicitly requested right now: remove auth code from legacy-version/.

Design Decisions

Keep the “legacy UI” as a runnable baseline, but remove proprietary auth

- We are not trying to “modernize” the entire frontend in one go.
- We are aiming for a stable baseline where:
 - `pnpm dev:ui` works without Manus-specific env vars,
 - pages render,
 - the Go backend API can be exercised without login redirects.

Treat “auth” as “environment glue” (not a core product feature) for now

Given the Go backend’s current behavior (synthetic admin user), the legacy UI’s OAuth code is not a core feature to preserve. It is an integration detail that blocks progress.

Defer “type import from legacy server” cleanup

The frontend’s `AppRouter` type is imported from `legacy-version/server/routers.ts`. Removing the server folder would break builds. A future cleanup should relocate shared API types into a dedicated shared package (or generate them), but that is not required to remove auth redirects and login URL construction.

Alternatives Considered

A1) Provide fake `VITE_OAUTH_PORTAL_URL` / `VITE_APP_ID` in local dev

Rejected: it keeps proprietary integration as a hard dependency, and it doesn’t address redirect UX or long-term cleanup.

A2) Make `getLoginUrl()` “safe” (return # if env missing) but keep auth flows

Rejected (for this ticket): the request is to remove auth code. Making it safe is a band-aid that still keeps Manus OAuth concepts spread throughout the UI.

A3) Delete legacy-version/ entirely

Rejected: we still need the UI and it currently provides the SPA assets. Also the frontend’s types import the legacy server router definition.

Implementation Plan

1. Inventory legacy UI structure and identify proprietary dependencies (done in Part 1/2 below).
2. Remove auth/OAuth concepts from the UI layer:
 - `getLoginUrl` and all call sites,
 - `redirect-to-login` handlers,
 - “Sign In Required” pages and gates,
 - `logout` handling that assumes cookies/portal login.
3. Ensure the UI still renders and can call `/api/trpc` against the Go server.
4. Follow-up (separate ticket):
 - Remove or isolate `vite-plugin-manus-runtime` if it is not required for non-Manus hosting.
 - Move `AppRouter` type out of `legacy-version/server/` to avoid keeping the legacy backend around just for TypeScript types.

Open Questions

- Do we want the UI to treat the user as “always admin” (for now) or to hide admin-only links entirely?
- Is `vite-plugin-manus-runtime` required for anything besides Manus hosting convenience?
- Should `auth.me` remain as a tRPC procedure (returning synthetic admin), or should we delete the auth router entirely and remove it from the client?

Part 1: Current Legacy Frontend Structure (What It Is, How It Builds, How It Runs)

1.1 Directory layout (`legacy-version/`)

At the top level, `legacy-version/` looks like a small monorepo:

- `legacy-version/client/`: the SPA (Vite root is `client/`)
- `legacy-version/server/`: an Express + tRPC server (historical; largely superseded by the Go backend)
- `legacy-version/shared/`: shared constants (cookie name, error messages)
- `legacy-version/drizzle/`: Drizzle schema + migrations (MySQL)
- `legacy-version/dist/`: build outputs (SPA goes to `dist/public`)
- `legacy-version/vite.config.ts`: Vite config for the SPA (root set to `client/`)

1.2 Toolchain & build system

The legacy frontend uses:

- Vite (with `@vitejs/plugin-react`)
- React 19
- TailwindCSS v4 + `tw-animate-css`
- `shadcn/ui` component scaffolding (Radix primitives + custom wrappers in `client/src/components/ui/*`)
- `@tanstack/react-query` + `@trpc/react-query` + `@trpc/client` (tRPC v11)
- `superjson` transformer (tRPC payload encoding)
- `wouter` for routing

Relevant build scripts (`legacy-version/package.json`):

- `pnpm dev:ui: vite (dev server)`

- `pnpm build:ui: vite build` (builds SPA into `legacy-version/dist/public`)

Vite configuration highlights (`legacy-version/vite.config.ts`):

- `root: legacy-version/client`
- Aliases:
 - `@` → `legacy-version/client/src`
 - `@shared` → `legacy-version/shared`
- `server.proxy["/api"]` → `http://127.0.0.1:8080` (the Go backend)
- Plugins:
 - React plugin
 - Tailwind plugin
 - `@builder.io/vite-plugin-jsx-loc`
 - `vite-plugin-manus-runtime` (proprietary integration surface)

1.3 Runtime architecture of the SPA

The SPA bootstraps in `client/src/main.tsx`:

- Creates a React Query `QueryClient`
- Creates a tRPC client using `httpBatchLink({ url: "/api/trpc", transformer: superjson })`
- Adds global query/mutation error listeners that redirect to login on “unauthorized” errors
- Renders `<App />` with both tRPC and React Query providers

This is a conventional “tRPC + React Query” architecture:

- tRPC procedures map to hooks like `trpc.documents.list.useQuery()`.
- Mutations map to `trpc.documents.create.useMutation()` etc.
- credentials: “include” is set on fetch; historically this was for cookie-based auth.

1.4 Pages and routing

Routing is managed by `wouter` in `client/src/App.tsx`:

- `/` → Home (document catalog)
- `/admin` → Admin (admin dashboard / document list)
- `/admin/new` and `/admin/edit/:id` → `DocumentEditor`
- `/admin/analytics/:id` → `Analytics` (document quiz analytics)
- `/admin/submissions` → `MySubmissions` (user submissions listing)
- `/documents/:slug` → `DocumentView` (read document, take quizzes)
- `/submission/:id` → `SubmissionReview` (review a submission, possibly with correct answers)

1.5 Core UI components/concepts

Key “product” components:

- `MarkdownRenderer`: renders markdown content, with syntax highlighting and GitHub-flavored markdown (`react-markdown` + `rehype/remark` plugins).
- `QuizForm`: renders quiz form fields based on parsed YAML DSL definitions (supports text, textarea, select, radio, checkbox, confirm).

Key “UX framework” components:

- `components/ui/*`: shadcn/ui wrappers around Radix primitives; provides consistent styling and variants.
- `ThemeContext`: small custom theme toggling based on adding/removing a `.dark` class.

1.6 Auth concept (as implemented today)

Auth is currently represented by:

- `client/src/const.ts`: `getLoginUrl()` builds a Manus OAuth URL from Vite env vars.
- `client/src/_core/hooks/useAuth.ts`: calls `trpc.auth.me` and returns `{ user, isAuthenticated, logout }`, with optional “redirect on unauthenticated”.
- Several pages render “Sign in required” UI and link to `getLoginUrl()`.
- The SPA global error handlers redirect to login if a tRPC error message matches `@shared/const.UNAUTHED_ERR_MSG`.

This is precisely the subsystem we are removing in this ticket.

Part 2: Component-by-Component Assessment (Simplify / Remove / Rewrite)

This section focuses on “what to keep vs. what to delete/replace” and why.

2.1 Vite plugins and Manus runtime

- `vite-plugin-manus-runtime` (rewrite/remove candidate)
 - Rationale: it is almost certainly Manus-hosting specific.
 - If we want a clean open-source-ish SPA, this should be optional or removed.
 - Action: confirm whether anything in `client/src/` requires it; if not, remove it and any related conventions.
- `@builder.io/vite-plugin-jsx-loc` (remove candidate)
 - Rationale: dev-only “jsx location” tooling; not needed for product functionality.
 - Action: can remove after verifying nothing depends on it.

2.2 Auth/OAuth subsystem (remove now)

Files/concepts:

- `client/src/const.ts` (`getLoginUrl`) — Manus OAuth portal URL generation
- `client/src/_core/hooks/useAuth.ts` — auth state hook
- global redirect-to-login behavior in `client/src/main.tsx`
- sign-in prompts in `pages/components`

Assessment:

- This is proprietary integration glue; it prevents local dev and is not needed given the current Go backend mode.
- It is spread across the app (pages + layout + bootstrap), so it should be removed comprehensively rather than partially.

Plan (this ticket; implemented):

- Remove `getLoginUrl` and all references.

- Remove redirect-to-login behavior.
- Replace `useAuth()` usages with a simpler “no-auth” approach (either: always-authenticated placeholder, or remove gates entirely).

2.3 Legacy server folder (keep for now, but mark for future removal)

The `legacy-version/server/` folder contains:

- Express app bootstrapping
- tRPC router definitions (`server/routers.ts`)
- Manus SDK integration (`server/_core/sdk.ts`, OAuth handlers, etc.)

Assessment:

- The Go backend supersedes the legacy Express server for runtime.
- However, the frontend imports `AppRouter` types from `server/routers.ts`, so deletion requires a type strategy (move types to `shared/`, generate types, or vendor minimal router type defs).

Plan (future ticket):

- Extract `AppRouter` types into a dedicated location not coupled to the legacy runtime.
- Then delete the Express server and Manus SDK integration.

2.4 Routing layer (keep)

wouter is lightweight and sufficient here. No immediate changes required.

2.5 tRPC + React Query client (keep, but simplify error handling)

Keep:

- tRPC v11 hooks + `httpBatchLink` with `superjson`

Simplify:

- Remove the special-case redirect behavior in the React Query cache subscriber (it’s part of the auth/OAuth coupling).

2.6 Markdown + quiz rendering (core product; keep)

Keep:

- `MarkdownRenderer`
- `QuizForm`
- `lib/widgets.ts` and `lib/presets.ts` (content authoring helpers in `DocumentEditor`)

Potential simplifications:

- Consolidate form definition parsing rules so both Go backend and frontend share the same structural expectations.
- Reduce UI component surface to only what is used by pages (many `components/ui/*` may be unused).

2.7 “Optional features” (likely remove unless actively used)

- AI chat UI (AIChatBox) — keep only if there is a corresponding Go backend endpoint; otherwise it’s dead weight.
- Google Maps component (Map.tsx) — likely remove if unused.
- Manus-specific modal (ManusDialog) — likely remove as part of auth removal.
- Component showcase page — remove if not part of product.

References

- Ticket diary: `markdown-quizz/ttmp/2026/01/19/M0-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/reference/01-diary.md`